

Senior Engineer

Interview Preparation Guide

65 Questions & Senior-Level Answers

API Design • Database & Schema • Distributed Systems • Performance
Software Design • Debugging • Principles & Patterns • Messaging

Gokulakonda Gautham • 2026

Table of Contents

1. API Design (13 questions)

- Q1. How do you design a REST API from scratch?
- Q2. How do you handle backward compatibility when evolving an API?
- Q3. How do you design pagination for a large dataset API?
- Q4. How do you design an API for rate limiting?
- Q5. What is the difference between REST, GraphQL, and gRPC — and when do you choose each?
- Q6. How do you design for idempotency in APIs?
- Q7. How do you design API authentication and authorization?
- Q8. How do you design an API that needs to support long-running operations?
- Q9. What is the difference between PUT and PATCH?
- Q10. How do you design API security beyond authentication?
- Q11. How do you design an API for file uploads?
- Q12. How do you design an idempotent job/background worker system?
- Q13. What is the difference between an idempotency token, correlation ID, and trace ID?

2. Database & Schema (9 questions)

- Q1. Walk me through database normalization — 1NF through 3NF with examples.
- Q2. How do you design a database schema for a multi-tenant SaaS application?
- Q3. How do you handle database migrations in production without downtime?
- Q4. Explain database indexing — when do you add an index and when is it harmful?
- Q5. What is the N+1 query problem and how do you solve it?
- Q6. What is a database transaction and what are isolation levels?
- Q7. What is a database deadlock and how do you prevent it?
- Q8. What is sharding and what are the common sharding strategies?
- Q9. How do database views, materialized views, and CTEs differ?

3. Distributed Systems (10 questions)

- Q1. Explain CAP theorem and how it affects your design decisions.
- Q2. What is the Saga pattern and how does it compare to 2-Phase Commit?
- Q3. How do you design a system to handle exactly-once message delivery?
- Q4. How do you implement distributed rate limiting across multiple instances?
- Q5. What is a circuit breaker and when do you use it?
- Q6. How do you design a distributed cache and what cache invalidation strategies do you know?
- Q7. What is service discovery and how does it work in Kubernetes?
- Q8. How do you implement graceful shutdown in a service?
- Q9. What is the outbox pattern and how does it solve dual writes?
- Q10. What is a health check? How should you design readiness vs liveness probes?

4. Performance & Latency (6 questions)

- Q1. How do you debug the latency of a slow API endpoint?
- Q2. How do you optimize a system that is hitting its throughput limit?
- Q3. How do you profile a Go application in production?
- Q4. What is connection pooling and why does it matter?
- Q5. What is tail latency and why is p99 more important than average latency?
- Q6. What is horizontal vs vertical scaling? When do you choose each?

5. Software Design (6 questions)

- Q1. How do you approach system design for a new feature from scratch?
- Q2. What is event-driven architecture and when would you use it?
- Q3. What is the difference between microservices and a monolith? When do you prefer each?
- Q4. How do you design a feature flag system?
- Q5. What is synchronous vs asynchronous communication in microservices?
- Q6. How do you handle secrets and configuration management in production?

6. Debugging (5 questions)

- Q1. How do you approach debugging a production issue you've never seen before?
- Q2. How do you debug a memory leak in a production Go service?
- Q3. How do you add observability to a microservices system?
- Q4. How do you approach a race condition in concurrent code?
- Q5. How do you debug an intermittent failure you cannot reproduce locally?

7. Principles & Patterns (10 questions)

- Q1. Walk me through SOLID principles with concrete examples.
- Q2. What are design patterns — explain 5+ patterns you've used in production.
- Q3. What is DRY vs WET vs AHA? When is code duplication acceptable?
- Q4. Explain CQRS and Event Sourcing.
- Q5. What are the 12-Factor App principles and how do they affect your architecture?
- Q6. What is dependency injection and why does it matter?
- Q7. What is eventual consistency and how do you handle it in the UI and backend?
- Q8. What is clean architecture / hexagonal (ports and adapters) pattern?
- Q9. What is immutability and why is it important in distributed systems?
- Q10. What is technical debt and when do you pay it down vs live with it?

8. Messaging & Events (6 questions)

- Q1. How does Kafka work and when do you choose it over RabbitMQ?
- Q2. How do you design a dead letter queue and why does it matter?
- Q3. Explain backpressure in streaming systems.
- Q4. How do you ensure message ordering in a distributed messaging system?
- Q5. What is database query plan caching and why can it cause problems?
- Q6. What is the difference between microservices synchronous and asynchronous communication and when do you use each?

1. API Design

Q1 How do you design a REST API from scratch?

API Design

Start with resources, not actions

Every noun in your domain becomes a URL segment. Operations map to HTTP verbs. Identify domain entities (User, Order, Product), use plural nouns like /users and /orders/:id/items, and avoid verbs in paths — /getUser is wrong; GET /users/:id is right.

Versioning strategy

- URL versioning (/v1/users) — most explicit, easiest to route at the gateway
- Header versioning (Accept: application/vnd.api+json;version=2) — cleaner URLs but harder to test in browser
- Prefer URL versioning for public APIs: cache-friendly and debuggable

Standard response shape

```
{ "data": { ... },           // always present on success
  "error": null,             // null on success, object on failure
  "meta": {                  // pagination, rate limit info
    "page": 1,
    "total": 240,
    "rate_limit_remaining": 95
  }
}
```

Status codes — the non-obvious ones

- 201 Created + Location header on POST
- 202 Accepted for async jobs (return a job ID)
- 422 Unprocessable Entity for business logic failures (valid JSON, invalid logic)
- 409 Conflict for duplicate resource creation
- Never let 500s leak internal details — map exceptions to problem types

Idempotency

PUT and DELETE must be idempotent by definition. For POST (e.g., payments), accept an Idempotency-Key header and cache the result for ~24h so retries are safe.

Q2 How do you handle backward compatibility when evolving an API?

API Design

Never break a contract that a client depends on. The key insight: evolution and breaking changes are different things.

Safe changes (non-breaking)

- Adding new optional fields to responses
- Adding new endpoints
- Adding new optional query parameters
- Relaxing validation rules

Breaking changes (require version bump)

- Renaming or removing a field
- Changing a field's type
- Making an optional field required
- Changing HTTP status codes for existing scenarios

Migration strategy

- Run v1 and v2 simultaneously (the expand-contract pattern)
- Deprecate v1 with a Deprecation and Sunset response header
- Use API gateways to route traffic gradually (feature flags at the gateway layer)

If you cannot version: introduce the new field alongside the old one, serve both, document the old one as deprecated, and remove after a sunset window.

Q3 How do you design pagination for a large dataset API?

API Design

Offset pagination

```
GET /users?page=5&limit=20
-- SQL: SELECT * FROM users ORDER BY id OFFSET 80 LIMIT 20
```

Problem: if a row is inserted between pages, you get duplicates or skips. Also, OFFSET scans are $O(\text{offset})$ — slow at large offsets.

Cursor-based pagination (preferred for real-time data)

```
GET /users?after=cursor_opaque_value&limit=20
-- Server decodes cursor to: WHERE id > 12345 ORDER BY id LIMIT 20
```

- Stable under inserts and deletes
- $O(\log n)$ with an index
- Tradeoff: cannot jump to page 50 arbitrarily

When to use which

- Offset: admin panels where you need arbitrary page jumps and data is not real-time
- Cursor/Keyset: feeds, timelines, any data with inserts during pagination

Q4 How do you design an API for rate limiting?

API Design

Common algorithms

- Token bucket: bucket of N tokens, refilled at rate R . Bursts allowed up to bucket size. Best for general use.
- Sliding window log: store timestamp of each request; count in last 60s. Accurate but memory-heavy.
- Fixed window counter: simple but has a spike problem at window boundary (2x traffic possible).
- Sliding window counter: hybrid — weight previous window by overlap fraction. Best accuracy-to-cost tradeoff.

What to limit on

- IP — but be careful with NAT (entire offices behind one IP)
- User/API key — better for authenticated endpoints
- User + endpoint combination — different limits for cheap vs expensive endpoints

Response contract

```
HTTP 429 Too Many Requests
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1720000000 // Unix timestamp
Retry-After: 60
```

Redis is the standard store. Use INCR + EXPIRE for fixed window; sorted sets for sliding window log. Use a Lua script to make INCR + check atomic.

Q5

What is the difference between REST, GraphQL, and gRPC — and when do you choose each?

API Design

REST

Resource-oriented, human-readable, stateless. Works great when your domain maps cleanly to nouns and you have diverse clients (web, mobile, third-party).

GraphQL

Query language — clients specify exactly what fields they need. Solves over-fetching and under-fetching. Best for: product APIs with many client types, rapid frontend iteration, or graph-shaped data models. Tradeoffs: harder to cache (POSTs), N+1 query problems require dataloaders, complex authorization logic.

gRPC

Binary protocol over HTTP/2 with Protobuf schemas. Best for: internal service-to-service communication, streaming, or strongly typed contracts across teams. Tradeoffs: not browser-native (needs grpc-web proxy), Protobuf schema management overhead.

Heuristic

- Public / third-party API → REST
- Complex product with many clients → GraphQL
- Internal microservices → gRPC

Q6

How do you design for idempotency in APIs?

API Design

Idempotency means calling an operation multiple times produces the same result as calling it once. Critical for payment, order creation, and any mutation.

HTTP verbs

GET, HEAD, PUT, DELETE are idempotent by spec. POST and PATCH are not by default.

Making POST idempotent

```
POST /payments
Idempotency-Key: uuid-from-client
```

```
key = hash("idempotency:" + client_id + ":" + idempotency_key)
cached = redis.get(key)
if cached: return cached

result = process_payment(body)
redis.setex(key, 86400, serialize(result))
return result
```

- Key must be scoped to the client
- Cache the entire response, not just a flag
- Use a distributed lock during processing to avoid double execution
- Return 200 (not 201) on a replayed request so clients know it was a replay

Q7 How do you design API authentication and authorization?

API Design

Authentication = who are you. Authorization = what can you do.

Authentication options

- API keys: simple for machine-to-machine. Hash them in DB (SHA-256), never store raw.
- JWT: stateless, self-contained. Access token (15min TTL) + refresh token (7-30d). Validate signature + expiry on every request. Never put sensitive data in payload.
- OAuth 2.0: for delegated access. Use authorization code flow with PKCE for web/mobile.

JWT access/refresh rotation

1. Login → issue access_token (15min) + refresh_token (7d, HttpOnly cookie)
2. Expired access_token → client sends refresh_token
3. Server validates, issues new access_token + rotates refresh_token
4. Old refresh_token is invalidated (token family detection)
5. Stolen refresh_token reused → invalidate entire token family

Authorization patterns

- RBAC (Role-Based): user has roles (admin, editor). Simple but coarse.
- ABAC (Attribute-Based): policy checks attributes. Fine-grained but complex.
- ReBAC (Relation-Based, like Google Zanzibar): user can edit doc because user is member of group with edit permission. Best for complex ownership graphs.

Q8 How do you design an API that needs to support long-running operations?

API Design

Never make the client wait synchronously for something that takes more than 1-2 seconds.

Async job pattern

```
// 1. Submit
POST /exports → 202 Accepted
{ "job_id": "job_abc123", "status_url": "/jobs/job_abc123" }

// 2. Poll
GET /jobs/job_abc123 → { "status": "running", "progress": 45 }
```

```
// 3. Done
GET /jobs/job_abc123 → { "status": "done", "result_url": "/exports/result.csv" }
```

Push-based alternatives

- Webhooks: server POSTs to a client URL when done. Best for async B2B integrations.
- SSE (Server-Sent Events): client opens a long-lived GET, server streams updates. Great for progress bars. Works through proxies unlike WebSockets.
- WebSockets: bidirectional, real-time. Overkill for one-way progress updates.

Job queue design

Store jobs in DB for durability. Track status transitions: pending → running → done/failed. Support retries with exponential backoff and a dead-letter queue.

Q9 What is the difference between PUT and PATCH?

API Design

PUT — full replacement

```
PUT /users/123
{ "name": "Gautham", "email": "g@example.com" }
// Result: user has exactly these fields — other fields are removed
```

PATCH — partial update

```
PATCH /users/123
{ "email": "new@example.com" }
// Result: only email is changed, name is preserved
```

Idempotency

PUT is idempotent by spec. PATCH is NOT necessarily idempotent — increment patches like { "count": "+1" } are not idempotent.

When to use which

- PUT: when the client knows and sends the complete representation
- PATCH: when clients update specific fields (inline editing, toggling a flag)
- Most modern APIs use PATCH for updates because clients rarely want to send the full resource

Q10 How do you design API security beyond authentication?

API Design

Input validation

- Validate every input at the API boundary: type, range, length, format
- Never trust client-supplied IDs to access resources without verifying ownership (prevents IDOR — Insecure Direct Object Reference)
- Parameterized queries always — never interpolate user input into SQL strings

Output filtering

- Do not return more fields than the caller needs
- Apply object-level and field-level authorization (not just route-level)

OWASP API Top 10 (key items)

- BOLA/IDOR: access control at the object level
- Mass assignment: do not bind request body fields to model fields blindly
- Excessive data exposure: return only what is needed
- Security misconfiguration: no default credentials, no verbose errors in prod, no open CORS

CORS

Do not set Access-Control-Allow-Origin: * for authenticated APIs. Whitelist specific origins. Use SameSite=Strict cookies to prevent CSRF.

Q11 How do you design an API for file uploads?

API Design

Small files (< 5 MB): direct upload

```
POST /uploads
Content-Type: multipart/form-data
// Body: file bytes + metadata fields
```

Simple but: gateway/load balancer body size limits, file goes through your API server consuming memory and CPU.

Large files: presigned URL pattern (preferred)

```
// Step 1: Client requests an upload URL
POST /uploads/initiate
{ "filename": "report.pdf", "size": 52428800 }
→ { "upload_url": "https://storage.googleapis.com/...", "file_id": "file_abc123" }

// Step 2: Client uploads DIRECTLY to GCS/S3
PUT upload_url ← binary file data

// Step 3: Client confirms upload
POST /uploads/file_abc123/confirm
```

Post-upload processing: do not process files synchronously. Publish an event; background workers handle virus scanning, format conversion, thumbnail generation.

Q12 How do you design an idempotent job/background worker system?

API Design

Background workers must be designed to handle reruns, duplicate job dispatches, and partial failures without corrupting state.

Core principle: jobs must be idempotent

Running the same job twice should produce the same result as running it once.

Design patterns

- Deduplicate at dispatch: check if a job with the same idempotency key already exists in pending/running state
- Check-before-act in the worker: verify the prerequisite state is still true before doing work
- Write result idempotently: use UPSERT / INSERT ON CONFLICT; use conditional updates (UPDATE ... WHERE status = 'pending')

Job state machine

```
pending → running → done
pending → running → failed → pending (retry) → ... → dead
```

Track attempt_count, last_error, next_run_at. Use exponential backoff for retry timing.

Q13

What is the difference between an idempotency token, correlation ID, and trace ID?

API Design

Idempotency token / key

Client-generated token passed with a mutation request. Enables the server to deduplicate retries. Scope: one logical operation (one payment, one order). Consumed by the server for deduplication logic.

Correlation ID

A request-scoped identifier that follows a request through all services it touches. Generated at the entry point. Passed via HTTP header (X-Correlation-ID). Every service logs it and passes it downstream. Purpose: trace all log lines across services that belong to one logical request.

Trace ID (OpenTelemetry)

Similar to correlation ID but with more structure: trace ID identifies the entire request tree; span IDs identify individual operations within the trace; parent-child span relationships form the trace tree. Propagated via W3C traceparent header.

In practice

- Idempotency keys: payments, order creation, any POST that creates something expensive
- Correlation IDs: log aggregation across services (grep in Loki/Elasticsearch)
- Trace IDs: latency debugging — which service is slow?

2. Database & Schema

Q14

Walk me through database normalization — 1NF through 3NF with examples.

Database

Normalization eliminates data anomalies by organizing data into smaller, well-defined tables.

1NF — First Normal Form

Rule: each column holds atomic (indivisible) values; no repeating groups.

```
-- BAD: items = "Pen, Notebook, Stapler" — multi-valued, non-atomic
orders(id, customer, items)

-- GOOD:
orders(id, customer_id)
order_items(order_id, item_id, quantity)
```

2NF — Second Normal Form

Rule: 1NF AND every non-key column must depend on the ENTIRE primary key (no partial dependencies). Only applies when PK is composite.

```
-- BAD: product_name depends only on product_id, not (order_id, product_id)
order_items(order_id, product_id, quantity, product_name)

-- GOOD: move product_name to products table
order_items(order_id, product_id, quantity)
products(product_id, product_name, price)
```

3NF — Third Normal Form

Rule: 2NF AND no transitive dependencies (non-key column depending on another non-key column).

```
-- BAD: dept_name depends on dept_id, not on emp_id
employees(emp_id, dept_id, dept_name)

-- GOOD:
employees(emp_id, dept_id)
departments(dept_id, dept_name)
```

When to denormalize

Normalization is for write correctness. Denormalize for read performance: pre-aggregated stats tables, materialized views, duplicating a rarely-changing column to avoid joins in hot read paths.

Q15

How do you design a database schema for a multi-tenant SaaS application?

Database

Pattern 1: Shared schema (row-level tenancy)

Add a tenant_id column to every table. Use PostgreSQL Row Level Security (RLS) to enforce isolation at the DB level.

```
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
CREATE POLICY tenant_isolation ON orders
  USING (tenant_id = current_setting('app.tenant_id')::UUID);
```

Pros: cheapest, single schema to migrate. Cons: one misconfigured query leaks all tenants' data.

Pattern 2: Schema-per-tenant

Each tenant gets their own PostgreSQL schema. Connection pool routes via SET search_path TO tenant_abc. Strong isolation, easy per-tenant migrations. More management overhead.

Pattern 3: Database-per-tenant

Complete isolation. Required for compliance (SOC2, HIPAA). High cost. Use for enterprise tiers.

Recommendation

Shared schema + RLS for startup/growth stage. Schema-per-tenant when you hit compliance requirements. Database-per-tenant only for regulated/enterprise.

Q16

How do you handle database migrations in production without downtime?

Database

The key is the expand-contract pattern (also called blue-green migrations).

The 3 phases

```
Phase 1 - EXPAND (backward compatible):
- Add the new column/table as nullable
- Deploy code that writes to BOTH old and new column
- Backfill existing rows in batches (never in one transaction)
```

```
Phase 2 - MIGRATE:
- Verify backfill is complete
- Switch reads to the new column
```

```
Phase 3 - CONTRACT:
- Drop old column/table after you are confident
```

Concrete example: renaming a column

```
-- Step 1: add nullable column (no lock contention)
ALTER TABLE users ADD COLUMN full_name TEXT;

-- Step 2: backfill in batches
UPDATE users SET full_name = first_name || ' ' || last_name
WHERE id BETWEEN 0 AND 10000 AND full_name IS NULL;

-- Step 3: add NOT NULL AFTER backfill
ALTER TABLE users ALTER COLUMN full_name SET NOT NULL;

-- Step 4: drop old columns after new code is stable
```

Never do in production

- DROP COLUMN or RENAME COLUMN with running traffic
- Adding a NOT NULL column with no default (table rewrite = full lock)

- Migrations inside application startup code

Q17

Explain database indexing — when do you add an index and when is it harmful?

Database

An index is a separate data structure (usually B-tree) that lets the DB find rows without a full table scan.

When to add an index

- Columns in WHERE clauses that filter significantly (high cardinality)
- Columns used in JOINS (foreign keys)
- Columns in ORDER BY + LIMIT (the index already has sorted order)
- Composite index for multi-column filters — put equality column first, range column last

Composite index column order

```
-- Query: WHERE tenant_id = ? AND created_at > ?
-- Correct: INDEX ON (tenant_id, created_at)
-- Wrong:   INDEX ON (created_at, tenant_id) - DB cannot use it efficiently
```

When an index is harmful

- Tables with very high write throughput — every INSERT/UPDATE/DELETE must update all indexes
- Low-cardinality columns (e.g., status with 3 values) — a full table scan may be faster
- Small tables — optimizer will choose seq scan anyway

PostgreSQL-specific

- Partial indexes: WHERE deleted_at IS NULL — only index active rows
- Covering indexes: INCLUDE (email) — avoid a heap fetch entirely
- EXPLAIN ANALYZE is your friend — look at actual vs estimated row counts

Q18

What is the N+1 query problem and how do you solve it?

Database

N+1 happens when you fetch N records and then execute 1 additional query per record to fetch related data.

```
users = db.query("SELECT * FROM users LIMIT 100") // 1 query
for user in users:
    orders = db.query("SELECT * FROM orders WHERE user_id = ?", user.id) // N
queries
// Total: 101 queries for 100 users
```

Solutions

- JOIN: fetch users and orders in a single query. Works well for 1:1 or small 1:many.
- Batch loading: collect all user IDs, do one WHERE user_id IN (...) query, then map in-app. This is what DataLoader (GraphQL) does.

```
user_ids = [u.id for u in users]
orders = db.query("SELECT * FROM orders WHERE user_id = ANY(?)", user_ids)
orders_by_user = group_by(orders, lambda o: o.user_id)
```

- Eager loading: ORMs like GORM/Sequelize have preload/include that do the batching automatically.

Detection

Log slow query count per request. If you see 100+ queries for a single endpoint, it is almost certainly N+1. Tools like pganalyze or query-count middleware help catch this in CI.

Q19 What is a database transaction and what are isolation levels?

Database

A transaction is a unit of work that is Atomic, Consistent, Isolated, and Durable (ACID).

The 4 isolation levels

- Read Uncommitted: can read uncommitted changes. Dirty reads possible. Never use.
- Read Committed: only reads committed data. Default in PostgreSQL. Prevents dirty reads but allows non-repeatable reads.
- Repeatable Read: a row read twice in the same transaction returns the same value. PostgreSQL's RR also prevents phantoms via MVCC.
- Serializable: transactions execute as if they ran one after another. Highest isolation, highest contention. Use for financial operations.

Read phenomena

Dirty Read:	T1 reads T2's uncommitted write
Non-repeatable:	T1 reads row, T2 updates, T1 reads again – different result
Phantom Read:	T1 reads set of rows, T2 inserts matching row – new row appears

Practical choice

- Read Committed: most CRUD operations
- Repeatable Read: generating consistent reports or snapshots
- Serializable: bank transfers, inventory deduction, any "check then act" pattern

Q20 What is a database deadlock and how do you prevent it?

Database

A deadlock occurs when two or more transactions are each waiting for a lock held by the other, and neither can proceed.

Transaction A:		Transaction B:
1. LOCK orders WHERE id = 1	←	1. LOCK payments WHERE order_id = 1
2. Waiting for payments lock		2. Waiting for orders lock
DEADLOCK		

Prevention strategies

- Consistent lock order: always acquire locks in the same order across all transactions
- Keep transactions short: the shorter a transaction holds locks, the less chance of conflict
- Use SELECT FOR UPDATE to acquire locks early, rather than upgrading a read lock mid-transaction

Optimistic locking

```
SELECT id, version FROM accounts WHERE id = 123;
-- do work ...
UPDATE accounts SET balance = new_balance, version = version + 1
WHERE id = 123 AND version = original_version;
-- 0 rows updated → someone else changed it → retry
```

Q21

What is sharding and what are the common sharding strategies?

Database

Sharding splits data across multiple databases (shards) to scale writes and storage beyond what a single node can handle. Only after exhausting: vertical scaling, read replicas, caching, query optimization.

Sharding strategies

- Range sharding: shard by value range (users 1-1M on shard 1). Simple, good for range queries. Problem: hotspots on recent data.
- Hash sharding: $\text{shard_id} = \text{hash}(\text{user_id}) \% \text{num_shards}$. Even distribution. Problem: range queries are expensive.
- Directory sharding: lookup table maps key → shard. Maximum flexibility. Problem: directory is a bottleneck.
- Consistent hashing: shards on a ring; keys map to nearest shard. Adding/removing a shard moves only $\sim 1/N$ keys. Used by Cassandra, Redis Cluster.

Shard key choice

Must be: high cardinality (not gender), uniformly distributed, present in most queries, immutable (or moving data between shards is expensive).

Cross-shard queries

The biggest pain point. Aggregate queries require scatter-gather (query all shards, merge results), or maintain a separate denormalized analytics DB.

Q22

How do database views, materialized views, and CTEs differ?

Database

View

A stored SQL query that you query like a table. No data is stored — every query against the view reruns the underlying SQL.

```
CREATE VIEW active_users AS
SELECT id, email FROM users WHERE deleted_at IS NULL;
```

Materialized View

Like a view, but the result is physically stored and periodically refreshed. Very fast reads. Tradeoff: data may be stale.

```
CREATE MATERIALIZED VIEW monthly_revenue AS
SELECT date_trunc('month', created_at) as month, sum(amount)
FROM orders GROUP BY 1;

REFRESH MATERIALIZED VIEW CONCURRENTLY monthly_revenue;
```

CTE (Common Table Expression)

A named subquery defined in the same statement using WITH. Scoped to the single query — not stored anywhere. Makes complex queries readable.

```
WITH ranked_users AS (  
    SELECT *, ROW_NUMBER() OVER (PARTITION BY country ORDER BY revenue DESC) as rank  
    FROM users  
)  
SELECT * FROM ranked_users WHERE rank <= 10;
```


3. Distributed Systems

Q23 Explain CAP theorem and how it affects your design decisions.

Distributed

CAP states that a distributed system can guarantee at most 2 of: Consistency, Availability, Partition Tolerance. Since network partitions will happen, the real choice is: during a partition, do you prefer Consistency (CP) or Availability (AP)?

CP systems

When a partition occurs, refuse to serve stale data. Return an error instead. Examples: ZooKeeper, etcd, traditional RDBMS with synchronous replication. Use when: financial systems, inventory deduction, leader election.

AP systems

When a partition occurs, continue serving — possibly returning stale data. Examples: Cassandra, DynamoDB, Redis cluster. Use when: user feeds, product catalogs, caches.

PACELC (the better model)

Even without partitions, there is a tradeoff between latency and consistency. PA/EL (Cassandra) vs PC/EC (Spanner-like systems).

Q24 What is the Saga pattern and how does it compare to 2-Phase Commit?

Distributed

2-Phase Commit (2PC)

Phase 1 – Prepare: coordinator asks all participants "can you commit?"
Phase 2 – Commit: if all say yes → commit; else → rollback

Problems: blocking protocol (coordinator failure leaves participants stuck), single point of failure, high latency.

Saga pattern

A saga is a sequence of local transactions, each publishing an event that triggers the next. If a step fails, compensating transactions undo previous steps.

```
Order Saga (choreography):
1. OrderService: create order → publish OrderCreated
2. PaymentService: charge card → publish PaymentSucceeded
   (on failure: publish PaymentFailed → OrderService cancels)
3. InventoryService: reserve stock → publish StockReserved
   (on failure: PaymentService refunds, OrderService cancels)
```

Two Saga styles

- Choreography: each service listens to events and reacts. Simple but hard to visualize.
- Orchestration: a central saga orchestrator directs each service and tracks state. Explicit, easier to debug. Preferred for complex flows.

Key insight: Sagas are eventually consistent — design your UI around this (show "payment pending" state, not a hard lock).

True exactly-once delivery at the infrastructure level is extremely hard. The practical solution is idempotent consumers.

Kafka's semantics

- At-most-once: auto-commit offsets before processing. Messages can be lost on crash.
- At-least-once: commit only after processing. Messages can be replayed → duplicates possible.
- Exactly-once: Kafka transactions + idempotent producer. Works end-to-end only within Kafka.

Idempotent consumer pattern (practical solution)

```
func processMessage(msg Message) error {
    exists, _ := redis.Get("processed:" + msg.ID)
    if exists { return nil } // already done

    err := doBusinessLogic(msg)
    if err != nil { return err } // will be retried

    redis.SetEX("processed:" + msg.ID, 86400, "1")
    return nil
}
```

Transactional outbox pattern

Write the event to an outbox table in the same DB transaction as the business data. A separate poller reads from the outbox and publishes to Kafka. Guarantees the event is published exactly once even if the app crashes mid-write.

Local in-memory rate limiting is useless when you have 10 instances — each instance only sees 1/10 of the traffic.

Centralized counter with Redis (Lua for atomicity)

```
local key = KEYS[1]
local now = tonumber(ARGV[1])
local window = tonumber(ARGV[2])
local limit = tonumber(ARGV[3])

redis.call('ZREMRANGEBYSCORE', key, 0, now - window)
local count = redis.call('ZCARD', key)
if count >= limit then return 0 end -- rate limited
redis.call('ZADD', key, now, now .. math.random())
redis.call('EXPIRE', key, window)
return 1 -- allowed
```

Sliding window with two counters

Keep a counter for the current window and previous window. Estimate current count as: $\text{prev_count} \times (1 - \text{elapsed/window}) + \text{curr_count}$. Gives an approximation without storing per-request timestamps.

Q27 What is a circuit breaker and when do you use it?

Distributed

A circuit breaker prevents cascading failures by detecting that a downstream service is unhealthy and stopping calls to it temporarily.

The three states

```
CLOSED (normal):
  Requests flow through. Track failure rate.
  If failures > threshold (e.g., 50% in 60s): → OPEN

OPEN (tripped):
  All requests fail immediately (no network call).
  Wait for reset_timeout (e.g., 30s): → HALF-OPEN

HALF-OPEN (probing):
  Allow limited test requests.
  If success: → CLOSED    If failure: → OPEN
```

Go implementation

```
cb := gobreaker.NewCircuitBreaker(gobreaker.Settings{
  Name:      "payment-service",
  MaxRequests: 3,
  Interval:   60 * time.Second,
  Timeout:    30 * time.Second,
  ReadyToTrip: func(counts gobreaker.Counts) bool {
    return counts.ConsecutiveFailures > 5
  },
})
```

Combine with: retry (exponential backoff for transient errors) + circuit breaker (persistent failures) + timeout (prevent indefinite blocking). These three form a resilience layer.

Q28 How do you design a distributed cache and what cache invalidation strategies do you know?

Distributed

Cache-aside (Lazy loading)

```
value = cache.get(key)
if not value:
    value = db.query(...)
    cache.set(key, value, ttl=300)
return value
```

Write-through

Every write to DB also writes to cache. Cache is always warm. Problem: write latency increases; cache filled with data that may never be read.

Write-behind (Write-back)

Write to cache only; async write to DB later. Best performance but risk of data loss if cache crashes.

Invalidation strategies

- TTL-based: simplest. Accept eventual consistency within the TTL window.
- Event-driven: on DB write, publish an invalidation event; cache subscriber deletes the key.
- Cache tags: associate keys with tags. Invalidate all keys with a tag atomically.

Cache stampede

When a hot key expires, thousands of requests hit the DB simultaneously. Solutions: probabilistic early expiration; mutex/lock on cache miss so only one request populates the cache.

Q29

What is service discovery and how does it work in Kubernetes?

Distributed

Client-side vs server-side

- Client-side: client queries a registry (Consul, etcd), gets a list of instances, picks one. Client does the work.
- Server-side: client sends to a single endpoint (load balancer); the LB discovers and routes. Client is dumb.

Kubernetes service discovery

```
Service: payment-service (ClusterIP: 10.96.0.10)
→ Routes to pods matching selector: app=payment-service

// DNS-based (automatic):
payment-service.default.svc.cluster.local:8080
// Any pod in the cluster can resolve this
```

CoreDNS resolves service names to ClusterIP. kube-proxy maintains iptables rules routing ClusterIP → pod IPs. Endpoints object tracks healthy pod IPs; updated when pods start/stop. Readiness probes ensure traffic only goes to ready pods.

In practice: just use Kubernetes Service names as hostnames. Your Go service connects to `http://payment-service:8080` — Kubernetes handles the rest.

Q30

How do you implement graceful shutdown in a service?

Distributed

Graceful shutdown means: stop accepting new work, finish in-flight work, then exit cleanly. Kubernetes sends SIGTERM before killing a pod.

Go implementation

```
func main() {
    server := &http.Server{Addr: ":8080", Handler: mux}
    go func() {
```

```

        if err := server.ListenAndServe(); err != http.ErrServerClosed {
            log.Fatal(err)
        }
    }()

    quit := make(chan os.Signal, 1)
    signal.Notify(quit, syscall.SIGTERM, syscall.SIGINT)
    <-quit

    ctx, cancel := context.WithTimeout(context.Background(), 30*time.Second)
    defer cancel()
    server.Shutdown(ctx)

    db.Close()
    kafkaProducer.Flush(5000)
    kafkaProducer.Close()
}

```

Kubernetes considerations

- Set terminationGracePeriodSeconds in pod spec to match your shutdown timeout
- A short sleep after receiving SIGTERM helps because there is a propagation delay before pod is removed from endpoints
- For Kafka consumers: commit offsets of in-flight messages before shutdown; they will be reprocessed on restart (idempotent consumer handles this)

Q31 What is the outbox pattern and how does it solve dual writes?

Distributed

The dual write problem: you need to write to your DB AND publish an event to Kafka atomically. If your process crashes between the two writes, you have inconsistency.

The wrong approach

```

// BAD: non-atomic dual write
db.Exec("UPDATE orders SET status = 'confirmed'")
kafka.Publish("order.confirmed", orderEvent) // crashes here? event never sent

```

Outbox pattern

```

// GOOD: write event to outbox table in SAME transaction
BEGIN;
    UPDATE orders SET status = 'confirmed' WHERE id = ?;
    INSERT INTO outbox (id, topic, payload, created_at)
    VALUES (uuid(), 'order.confirmed', json(orderEvent), now());
COMMIT;

// Separate process: polls outbox, publishes to Kafka, then deletes row

```

Alternative: CDC (Change Data Capture)

Debezium reads the PostgreSQL WAL and publishes every DB change as a Kafka event. No outbox table needed — the transaction log IS the outbox. Powerful but more infrastructure.

Liveness probe

"Is this process alive? Should Kubernetes restart it?" If it fails: container is killed and restarted.

```
GET /health/live → 200 OK { "status": "ok" }  
// Only check: can the HTTP server handle a request?  
// Do NOT check DB connectivity – a DB outage should not restart all pods
```

Readiness probe

"Is this instance ready to receive traffic?" If it fails: pod is removed from service endpoints, no new requests sent. Container is NOT restarted.

```
GET /health/ready  
// Check: DB connection works, Redis connected, cache warmed up  
→ 503 if not ready  
→ 200 OK when ready
```

Design guidelines

- Liveness: return 200 if the process is alive. Do not check dependencies.
- Readiness: check critical dependencies with short timeouts. Return 503 with detail if critical dependency is down.
- Never make readiness dependent on non-critical external services

4. Performance & Latency

Q33 How do you debug the latency of a slow API endpoint?

Performance

Step 1: Reproduce and isolate

- Is it slow for all users or specific users/tenants?
- Did latency spike after a deploy (code change) or gradually (data growth)?
- Is it consistently slow or intermittently slow?

Step 2: Distributed tracing

```
Request (total: 2100ms)
├── Middleware auth check      (8ms)
├── DB query 1: fetch user     (12ms)
├── DB query 2: fetch orders  (1800ms) ← HOT SPOT
├── External API call         (200ms)
└── Response serialization    (80ms)
```

Step 3: Database query analysis

- EXPLAIN ANALYZE the slow query
- Look for: Seq Scan on large table, sort without index, high actual vs estimated rows
- Check pg_stat_statements for query patterns and total time
- Check for lock waits: pg_locks, pg_blocking_pids()

Step 4: Infrastructure layer

- Network latency between services (especially cross-AZ calls)
- Connection pool exhaustion (requests queuing for a connection)
- CPU throttling in Kubernetes (CPU limits cause invisible throttling)

Step 5: Application code

- N+1 queries (log query count per request)
- Synchronous calls that could be parallelized
- Large JSON serialization (benchmark with pprof)

Q34 How do you optimize a system that is hitting its throughput limit?

Performance

First, find the bottleneck (Little's Law: throughput = concurrency / latency). Do not optimize before profiling.

Identify the bottleneck

- CPU-bound: CPU at 100%, latency scales linearly with load
- I/O-bound: high wait time, CPU low; look at DB, network, disk
- Concurrency-bound: connection pool exhausted, thread pool full

CPU-bound solutions

- Horizontal scaling (more instances)
- Profile hot functions (pprof in Go) and optimize algorithms
- Reduce serialization overhead (use protobuf instead of JSON for internal calls)

I/O-bound solutions

- Cache frequently read data (Redis)
- Use read replicas for read-heavy workloads
- Batch writes (buffer and flush vs one-by-one inserts)

Architecture-level

- Move expensive work to async (publish to Kafka, process in background)
- Shard by tenant/user ID to distribute load
- Use a CDN for static assets and cacheable API responses

Q35 How do you profile a Go application in production?

Performance

Go has excellent built-in profiling via pprof.

Expose pprof endpoint

```
import _ "net/http/pprof"
go func() {
    log.Fatal(http.ListenAndServe(":6060", nil))
}()
```

Common profiles

```
go tool pprof http://localhost:6060/debug/pprof/profile?seconds=30 // CPU
go tool pprof http://localhost:6060/debug/pprof/heap              // Memory
go tool pprof http://localhost:6060/debug/pprof/goroutine         // Goroutine
leaks
go tool pprof http://localhost:6060/debug/pprof/block            // Blocking
go tool pprof http://localhost:6060/debug/pprof/mutex            // Mutex
contention
```

In pprof interactive mode

```
(pprof) web // opens SVG flame graph in browser
(pprof) top10 // top 10 functions by CPU/mem
(pprof) list FuncName // annotated source
```

CPU profiling has ~5% overhead. Short (5-10s) profiles are safe in prod. For continuous profiling, use Pyroscope or Datadog's continuous profiler.

Q36 What is connection pooling and why does it matter?

Performance

Creating a new DB connection is expensive: TCP handshake, authentication, SSL negotiation — can take 20-50ms. A connection pool reuses existing connections.

How it works

```
Pool initialization:
  min connections: pre-created, always ready
  max connections: cap on total concurrent connections
```


Request flow:

1. App requests a connection from pool
2. Idle connection available: return immediately
3. Pool not full: create new connection
4. Pool full: queue request (with timeout)
5. After use: return to pool (do not close)

PgBouncer modes

- Session mode: connection assigned for entire client session. No benefit for short queries.
- Transaction mode: connection assigned per transaction only. Best multiplexing. Note: prepared statements and advisory locks do not work across transactions.

Scale impact

PostgreSQL has a process per connection. With PgBouncer: 5000 app connections → 100 real DB connections. In Go: set `db.SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`.

Q37

What is tail latency and why is p99 more important than average latency?

Performance

The problem with averages

If 99% of requests take 10ms and 1% take 10 seconds, your average is ~110ms — looks fine. But 1% of requests means millions of users at scale.

Percentile latencies

- p50: median — half of requests are faster than this
- p95: 95% of requests are faster than this
- p99: 99% of requests are faster than this
- p999: 99.9% — critical for financial/safety systems

Why tail latency matters more at scale

In a system where a user request fans out to 100 microservices, the user waits for the SLOWEST response. If each service has 1% chance of being slow, the probability that at least one is slow = $1 - (0.99)^{100} \approx 63\%$.

Common causes of high tail latency

- GC pauses
- Lock contention — some requests wait while others hold a lock
- Hot partitions — some DB rows or cache keys get much more traffic
- Connection pool exhaustion — unlucky requests queue for a connection
- CPU throttling in Kubernetes (CPU limits cause invisible throttling)

Hedged requests

If a request to a backend has not returned within p95 time, send a duplicate request to a second instance. Accept whichever returns first. Cancel the other. Reduces p99 at the cost of slightly higher backend load.

Q38

What is horizontal vs vertical scaling? When do you choose each?

Performance

Vertical scaling (scale up)

Give the existing server more resources. Simpler — no application changes needed. Pros: no distributed systems complexity. Cons: hardware limits, single point of failure, requires downtime to resize.

Horizontal scaling (scale out)

Add more instances of the service. Requires the application to be stateless. Pros: theoretically unlimited scale, redundancy. Cons: distributed systems complexity, stateful components (DB, cache) become bottlenecks.

Database horizontal scaling

- Read replicas: scale reads horizontally
- Sharding: partition data across multiple primaries — high complexity, only when vertical + replicas are not enough
- CQRS with a separate read store (Elasticsearch, Redis) for specific query patterns

5. Software Design

Q39

How do you approach system design for a new feature from scratch?

Sys Design

1. Clarify requirements (the most important step)

- What is the user/system goal? What problem does this solve?
- Who are the users and what scale do we expect? (DAU, QPS, data volume)
- What are the latency/consistency/availability requirements?
- What is explicitly in scope and out of scope?

2. Define the data model first

The data model is the foundation. If you get it wrong, everything built on it is wrong. Identify entities, relationships, and access patterns (read-heavy vs write-heavy, hot keys, fan-out).

3. Design the API contract

Before the internals. This is the interface your teammates depend on.

4. High-level architecture

Draw the boxes: clients, API gateway, services, queues, databases, caches. Identify data flows.

5. Deep dive on the hardest part

Every system has one tricky piece — scaling writes, real-time updates, consistency across services. Spend most design time here.

6. Failure modes

- What happens if service X goes down?
- What happens if we lose a message?
- What happens if we get a duplicate request?

Q40

What is event-driven architecture and when would you use it?

Sys Design

Core concepts

- Event: an immutable record of something that happened (OrderPlaced, PaymentFailed)
- Producer: publishes events without knowing who consumes them
- Consumer: subscribes to event types and reacts
- Broker: Kafka, RabbitMQ, Google Pub/Sub — the durable channel between them

When to use it

- Decoupling services that do not need immediate responses
- Fan-out: one event triggers many consumers (order placed → notify, invoice, fulfillment)
- Audit logs and event sourcing
- Handling traffic spikes — buffer in the queue, consumers process at their own pace

When NOT to use it

- When you need an immediate response (login, checkout confirmation)
- Simple CRUD apps — event-driven adds complexity without benefit

Tradeoffs to call out

Eventual consistency. Harder to trace a request across services (need correlation IDs). Schema evolution must be backward-compatible. Testing is harder — you need to verify events were published with the right content.

Q41

What is the difference between microservices and a monolith? When do you prefer each?

Sys Design

Monolith

Single deployable unit. All modules share a process and database. Pros: simple to develop, test, deploy, debug; low latency; ACID transactions across the entire domain. Cons: scales as a unit, deployment risk for unrelated changes, team coupling.

Microservices

Each service owns its domain, database, and deployment. Pros: independent scaling, deployment, team autonomy, technology flexibility. Cons: distributed systems complexity, network latency, no cross-service ACID transactions, operational overhead.

Honest recommendation

Start with a modular monolith. Extract services when you have: (a) a team ownership boundary, (b) a scaling bottleneck, (c) a compliance boundary. Do not microservice-ify prematurely.

Modular monolith

Strict module boundaries within one codebase and one deployment. Each module owns its schema (separate tables, no cross-module joins). Extract as a service later when justified — the module boundary already gives you the interface.

Q42

How do you design a feature flag system?

Sys Design

Core data model

```
flags:      id, name, description, enabled (global kill switch)
flag_rules: flag_id, rule_type, value, percentage
            // rule_type: user_id, tenant_id, percentage_rollout
```

Evaluation logic

```
func IsEnabled(flagName string, ctx EvalContext) bool {
    flag := getFlag(flagName)
    if !flag.Enabled { return false }
    for _, rule := range flag.Rules {
        switch rule.Type {
        case "user_id":
            if ctx.UserID == rule.Value { return true }
        case "percentage":
            // Stable: same user always gets same result
            bucket := stableHash(ctx.UserID, flagName) % 100
```

```

        if bucket < rule.Percentage { return true }
    }
}
return false
}

```

Key design decisions

- Stable bucketing: same user always gets the same flag value — use a hash, not random
- Cache flags: flags are read on every request. Cache in memory with a short TTL (30s).
- Gradual rollout: start at 1%, watch error rates, then 10%, 50%, 100%
- Kill switch pattern: flag is on by default; flip it off in incidents

Q43

What is synchronous vs asynchronous communication in microservices?

Sys Design

Synchronous (request-response)

Service A calls Service B and waits for a response. HTTP/REST, gRPC. Pros: simple, immediate consistency, easy error handling. Cons: temporal coupling (B must be up for A to work), cascading failures.

Asynchronous (message-based)

Service A publishes an event; Service B processes it later. Kafka, RabbitMQ. Pros: temporal decoupling, better resilience, absorbs traffic spikes. Cons: eventual consistency, harder to debug, complex error handling.

When to use which

- Sync: login, checkout confirmation, user-facing queries that need immediate answers
- Async: sending emails, updating analytics, syncing to external systems

Hybrid: async request-response

POST /export returns 202 + job_id. Client polls or subscribes to SSE for completion. Best of both worlds.

Q44

How do you handle secrets and configuration management in production?

Sys Design

Never do

- Hardcode secrets in source code (they end up in git history permanently)
- Commit .env files with real values
- Pass secrets as Docker CMD arguments (visible in docker ps and logs)

Secret management options

- Kubernetes Secrets: base64-encoded (not encrypted by default). Enable encryption at rest via KMS.
- GCP Secret Manager / AWS Secrets Manager / HashiCorp Vault: secrets stored encrypted, accessed via API with IAM authentication. Support secret rotation, audit logs, versioning.

- External Secrets Operator: syncs secrets from Vault/GCP Secret Manager into Kubernetes Secrets automatically.

Principle of least privilege

Each service should only have access to the secrets it needs. Do not give the frontend service access to the DB password.

Secret rotation

- Design services to reload secrets without restart (poll Secret Manager periodically)
- Use short-lived credentials where possible (GCP Workload Identity, AWS IAM Roles for Service Accounts)
- Configuration vs secrets: non-sensitive config (feature flags, URLs, timeouts) goes in ConfigMaps. Secrets go in a secret store. Never conflate them.

6. Debugging

Q45

How do you approach debugging a production issue you've never seen before?

Debugging

1. Contain first, fix second

If users are impacted: can I roll back? Can I feature-flag this off? Mitigation first, root cause second.

2. Gather evidence

- When did it start? (correlate with deploys, config changes, traffic spikes)
- How many users/requests are affected? What is the blast radius?
- What changed? (deployment, config, traffic pattern, upstream dependency)
- What do the logs say? Look for the FIRST error, not the cascade that follows.

3. Form a hypothesis

Based on evidence, form ONE specific hypothesis. Do not fix five things at once — you will not know what worked.

4. Test the hypothesis

Find evidence that confirms or refutes it WITHOUT making changes first. Only then make one targeted fix.

5. Fix, verify, document

Make one targeted change. Monitor for 10 minutes. Write a postmortem: what happened, why, what we changed, what prevents it in future. The postmortem is as valuable as the fix.

Q46

How do you debug a memory leak in a production Go service?

Debugging

Signs of a memory leak

- Memory grows monotonically over hours/days and never releases
- Eventually OOMKilled by Kubernetes

Step 1: Confirm it is a leak

Plot memory usage over time. If it grows unboundedly, it is a leak. If it is high but stable, it is a sizing issue.

Step 2: Heap profile

```
go tool pprof http://localhost:6060/debug/pprof/heap

(pprof) top          // what is holding most memory?
(pprof) list PackageName.FunctionName
```

Common causes in Go

- Goroutine leak: goroutines that never exit (blocked on channel with no writer/reader). Check /debug/pprof/goroutine — if count grows, you have a goroutine leak. Fix: always pass context with cancellation.

- Unbounded cache: in-memory map that grows forever. Fix: add eviction (LRU cache, TTL).
- HTTP response body not closed: `resp.Body.Close()` must always be called.
- Timer not stopped: `time.After` in a loop creates a new timer every iteration. Use `time.NewTimer` and `Reset/Stop`.
- Reference held in slice: `s = s[1:]` does not release the backing array.

Q47 How do you add observability to a microservices system?

Debugging

Logs — the "what happened"

- Structured logging (JSON) always — `grep` on free text does not scale
- Every log entry: timestamp, level, service, `trace_id`, `span_id`, message + structured fields
- Log levels: ERROR (page-worthy), WARN (investigate soon), INFO (normal events), DEBUG (dev only)

```
logger.Info("payment processed",
    "payment_id", p.ID,
    "amount", p.Amount,
    "tenant_id", p.TenantID,
    "duration_ms", elapsed.Milliseconds())
```

Metrics — the "how much/how often"

- RED method: Rate (requests/sec), Errors (error rate), Duration (latency percentiles)
- Track p50, p95, p99 latency — not just averages (averages hide tail latency)
- Expose Prometheus metrics at `/metrics`; scrape with Prometheus, visualize in Grafana

Traces — the "where did time go"

- OpenTelemetry SDK: instrument at service entry/exit and before/after external calls
- Propagate trace context via HTTP headers (W3C traceparent)
- Export to Jaeger, Tempo, or DataDog

Alerts

Alert on symptoms (user impact), not causes. Error rate > 1% is a symptom. High CPU is a cause — usually not worth waking someone up for.

Q48 How do you approach a race condition in concurrent code?

Debugging

Detection

```
go test -race ./...
go run -race main.go
```

Common patterns and fixes in Go

```
// RACE: multiple goroutines write to map
var cache = make(map[string]int)
go func() { cache["key"] = 1 }()
go func() { cache["key"] = 2 }()
```



```
// FIX: use sync.RWMutex or sync.Map
var mu sync.RWMutex
mu.Lock()
cache["key"] = value
mu.Unlock()
```

- Check-then-act: read a value, decide to act, but value changes between read and act. Fix: hold the lock across the check AND the act.
- Closing a channel twice. Fix: only the sender closes channels; use sync.Once if multiple closers are possible.

Goroutine variable capture

```
// RACE: loop variable captured by goroutine
for _, v := range items {
    go func() { process(v) }() // v is shared
}

// FIX: shadow with local copy
for _, v := range items {
    v := v
    go func() { process(v) }()
}
```

Q49

How do you debug an intermittent failure you cannot reproduce locally?

Debugging

Step 1: Make it observable in production

- Add detailed logging around the failure path: log inputs, intermediate state, and outputs
- Add a metric counter for every branch of the code path
- Add distributed trace spans so you can see exactly where time is spent when it fails

Step 2: Characterize the failure

- What % of requests fail? Is it time-based (only at night, only at peak)?
- Is it correlated with a specific user, tenant, data shape, or request pattern?
- Does it fail in one instance but not others? (State in one pod)
- Is it correlated with another event (another service timing out, a batch job running)?

Step 3: Reproduce in controlled environment

- Replay the exact input from logs against a staging environment
- Add fault injection: artificially slow down or fail dependencies to reproduce race conditions
- Write a load test that approximates production traffic patterns

Step 4: Hypothesis and verification

Form one hypothesis. Add an assertion or log that would definitively confirm or deny it. Deploy and wait for the next occurrence. This is often the only way to debug true intermittent failures.

7. Principles & Patterns

Q50 Walk me through SOLID principles with concrete examples.

Principles

S — Single Responsibility

A class/module should have one reason to change. Split UserService into AuthService, EmailService, and UserRepository — each has one job.

O — Open/Closed

Open for extension, closed for modification. Define a PaymentProcessor interface; each payment provider implements it. Adding a new payment type = add a new struct, no modification to existing code.

L — Liskov Substitution

Subtypes must be substitutable for their base types without breaking behavior. Classic violation: Square "is-a" Rectangle but overriding SetWidth breaks the invariant that width and height are independent.

I — Interface Segregation

Clients should not depend on methods they do not use. Split a fat Worker interface (Work, Eat, Sleep) into Worker (Work) and Human (Work, Rest). A Robot should not be forced to implement Eat.

D — Dependency Inversion

```
// BAD: service directly instantiates DB
type OrderService struct { db *PostgresDB }

// GOOD: depend on interface
type OrderRepository interface {
    FindByID(id string) (*Order, error)
}
type OrderService struct { repo OrderRepository }
```

Q51 What are design patterns — explain 5+ patterns you've used in production.

Principles

Repository pattern

Abstracts data access. Your service talks to an interface; the concrete implementation hits PostgreSQL or a mock in tests.

Factory pattern

```
func NewProcessor(cfg Config) PaymentProcessor {
    switch cfg.Provider {
        case "stripe": return NewStripeProcessor(cfg.APIKey)
        case "razorpay": return NewRazorpayProcessor(cfg.Secret)
    }
    return nil
}
```

Observer / Pub-Sub pattern

Publishers emit events; subscribers react. Decouples producers from consumers. Kafka is pub-sub at infrastructure scale. In-process: Go channels implement this naturally.

Strategy pattern

Define a family of algorithms, encapsulate each, make them interchangeable. Used in test automation tools — different script generation strategies per browser type, swappable without changing the caller.

Circuit Breaker pattern

State machine around a remote call that trips open when failure rate exceeds threshold. Prevents cascading failures.

Outbox pattern

Reliably publish events by writing to an outbox table in the same DB transaction as business data. A poller reads and publishes. Guarantees at-least-once delivery.

BFF (Backend for Frontend) pattern

Each client type (mobile, web, third-party) has its own API layer that aggregates and tailors responses. Avoids exposing internal microservice complexity to clients.

Q52

What is DRY vs WET vs AHA? When is code duplication acceptable?

Principles

DRY — Don't Repeat Yourself

Every piece of knowledge should have a single authoritative representation. Not just code — also configuration, documentation, schema definitions.

WET — Write Everything Twice

The observation that premature abstraction is worse than duplication. Wait until you have written it a third time before abstracting.

AHA — Avoid Hasty Abstractions

Prefer duplication over the wrong abstraction. A wrong abstraction forces every future use case into a shape it does not fit, making code harder to change, not easier.

When duplication is acceptable

- The two pieces of code look similar but change for different reasons — coincidentally similar, not the same thing
- You do not yet know what the abstraction should look like (wait for the third occurrence)
- The abstraction would require so many parameters that it becomes harder to understand than the duplication
- Tests — some duplication in tests is acceptable for readability and isolation

Duplication is better than coupling. If sharing requires importing one service into another that should not know about it, keep the duplication.

Q53

Explain CQRS and Event Sourcing.

Principles

CQRS — Command Query Responsibility Segregation

Separate the write model (commands) from the read model (queries).

```
Write side: optimized for consistency
PlaceOrderCommand → OrderAggregate → orders table (normalized)
```

```
Read side: optimized for read performance
OrdersQuery → orders view (denormalized, pre-joined)
→ or Elasticsearch index
```

Benefits: read and write sides can scale independently, read models shaped exactly to UI needs. Drawbacks: eventual consistency between models, more complexity to maintain.

Event Sourcing

Instead of storing current state, store every event that led to it.

```
// Traditional: store current balance
accounts: { id: 1, balance: 500 }

// Event sourcing: store events
events: [
  { type: "AccountOpened", amount: 1000 },
  { type: "Withdrawn", amount: 300 },
  { type: "Deposited", amount: -200 },
]
// Current state = replay all events
```

Benefits: complete audit trail, time-travel debugging, natural fit with CQRS. Drawbacks: replaying many events is slow (need snapshots), schema evolution for past events is hard.

Q54

What are the 12-Factor App principles and how do they affect your architecture?

Principles

Most impactful factors

- Factor III — Config in environment: no hardcoded config. Everything that differs between environments is in environment variables. This is what makes Docker + Kubernetes deployments clean.
- Factor IV — Backing services as attached resources: treat DB, Redis, Kafka as attached resources accessed via URL. Swap Postgres for RDS by changing an env var, not code.
- Factor VI — Stateless processes: processes share nothing. No in-memory session state. Session data goes in Redis. This is what makes horizontal scaling trivial.
- Factor IX — Disposability: fast startup, graceful shutdown. Handle SIGTERM, finish in-flight requests, release connections cleanly. Critical for Kubernetes rolling deployments.
- Factor X — Dev/prod parity: dev environment should mirror prod. Use Docker Compose to run Postgres, Kafka, Redis locally instead of sqlite or in-memory fakes.

Q55

What is dependency injection and why does it matter?

Principles

Dependency injection means a component receives its dependencies from outside rather than creating them internally.

```
// WITHOUT DI: creates its own dependency
type OrderService struct{}
func (s *OrderService) PlaceOrder(order Order) error {
    db, _ := sql.Open("postgres", "hardcoded_conn_string")
    ...
}

// WITH DI: receives dependency
type OrderService struct {
    repo OrderRepository
    mailer Mailer
}
func NewOrderService(repo OrderRepository, mailer Mailer) *OrderService {
    return &OrderService{repo: repo, mailer: mailer}
}
```

Why it matters

- Testability: inject mocks in tests, never need a real DB connection
- Flexibility: swap Redis for Memcached by providing a different implementation of the same interface
- Explicit dependencies: all dependencies visible in the constructor, no hidden globals

In Go, DI is done manually (wire up in main.go) or with tools like google/wire. Manual DI is preferred for smaller services — it is explicit and the compiler catches everything.

Q56

What is eventual consistency and how do you handle it in the UI and backend?

Principles

Where it comes from

- Kafka consumer processing lag
- Read replica replication lag
- Distributed caches with TTL
- Saga pattern — partial state between compensating transactions

Backend strategies

- Read-your-writes consistency: after a write, route subsequent reads from the same user to the primary for a short window (e.g., 5s), then fall back to replica.
- Monotonic reads: a user always reads from the same replica within a session. They will not see data go back in time.
- Conditional reads: return a version/etag with data. Client sends it back; if server is behind, it can fetch from primary.

UI strategies

- Optimistic updates: update the UI immediately on user action; roll back if the server returns an error.
- Show pending state: display "Submitting..." or a "pending" badge on items being processed asynchronously.
- Polling/SSE: poll the status endpoint or stream server-sent events until the operation is confirmed.

Q57

What is clean architecture / hexagonal (ports and adapters) pattern?

Principles

Both address the same problem: business logic should not depend on infrastructure (DB, HTTP, message queue). Infrastructure should depend on business logic.

Clean architecture

Outer → Inner (dependency direction: always inward)

[Frameworks & Drivers] → [Interface Adapters] → [Use Cases] → [Entities]

- Entities: pure business objects, no framework dependencies
- Use Cases: application-specific business rules
- Interface Adapters: controllers, repositories (implement ports)
- Frameworks: HTTP, DB, message brokers

Hexagonal / Ports and Adapters

```
// Port (defined by core):
type PaymentGateway interface {
    Charge(userID string, amount int) error
}

// Adapter (implements port):
type StripeAdapter struct { client *stripe.Client }
func (s *StripeAdapter) Charge(userID string, amount int) error { ... }

// Core does not import stripe - only knows about the port
```

You can swap Stripe for Razorpay by writing a new adapter. The business logic is framework-free and easily unit-testable.

Q58

What is immutability and why is it important in distributed systems?

Principles

Why it matters in distributed systems

- No cache invalidation needed: if a value never changes, cached copies are always valid
- Safe to share across goroutines/threads without locks: no mutation = no race conditions
- Reproducibility: given the same immutable inputs, a function always produces the same output
- Event logs are naturally immutable: a Kafka log is an append-only sequence of immutable events

In code

```
// Instead of mutating:
user.status = "active"

// Return a new value:
activeUser := User{...user, Status: "active"}
```

Content-addressable storage

Store data by hash of its content. If the content does not change, the hash does not change. Used by Docker image layers, Git (every commit is an immutable snapshot addressed by its hash).

Practical note: full immutability is not always practical for databases. The pattern is: make the mutation log immutable (event sourcing), even if the current state projection is mutable.

Q59

What is technical debt and when do you pay it down vs live with it?

Principles

Not all debt is equal

- Deliberate prudent debt: "We know this is not the right approach, but we will refactor later." Legitimate — document it.
- Deliberate reckless debt: "We do not have time for tests." Compounds fast.
- Inadvertent debt: "We did not know better at the time." Fix it when you touch the code (Boy Scout rule).

When to pay it down

- When it directly slows a current feature
- When it causes incidents (reliability debt is the most expensive kind)
- When onboarding new engineers takes more than 1 day to understand a subsystem
- When the area is touched frequently (high-traffic code paths get more ROI from cleanup)

When to live with it

- When the area is stable and rarely changed
- When fixing it would be a large refactor with high risk and low business impact
- When you are pre-product-market-fit (speed > cleanliness)

Track debt explicitly: a "tech debt" label in your issue tracker, a section in sprint planning for debt reduction (the 20% rule). Do not let it live only in engineers' heads.

8. Messaging & Events

Q60

How does Kafka work and when do you choose it over RabbitMQ?

Messaging

How Kafka works

- Topics are split into partitions; each partition is an ordered, immutable log
- Producers append to the end; consumers read at their own offset
- Offsets are committed by consumers — if a consumer crashes, it restarts from its last committed offset
- Consumer groups: each partition is consumed by exactly one consumer in the group, giving parallelism
- Messages are retained for a configurable period (e.g., 7 days) regardless of consumption — multiple consumer groups can independently read the same topic

Kafka vs RabbitMQ

- RabbitMQ: messages are deleted after consumption (queue semantics). Best for task queues where each message needs to be processed once.
- Kafka: messages are retained; multiple consumers can replay. Best for: event streaming, audit logs, event sourcing, data pipelines, fan-out to many consumers.

Partitioning strategy

- Use `tenant_id` or `user_id` as key if you need ordered processing per tenant/user
- Using a constant key = all messages go to one partition = no parallelism

Consumer group lag

The most important Kafka operational metric. Lag = latest offset - consumer offset. If lag is growing, your consumers are not keeping up. Scale out consumers (up to the number of partitions).

Q61

How do you design a dead letter queue and why does it matter?

Messaging

A dead letter queue (DLQ) captures messages that fail processing after all retries, so they do not block the main queue or get silently lost.

DLQ design

```
Main Topic → Consumer → ProcessMessage()
                        ↓ on failure
                    Retry (up to N times, exponential backoff)
                        ↓ still failing
                    Dead Letter Topic (DLQ)
                        ↓
                Alert + Dashboard for manual inspection
```

Retry strategy

- Distinguish between transient errors (DB connection timeout — retry) and permanent errors (invalid data format — DLQ immediately)

- Exponential backoff: 1s, 2s, 4s, 8s, ... up to max (30s)
- Jitter: add randomness to backoff to avoid thundering herd on recovery

DLQ operations

- Store original message + error metadata (exception, stack trace, attempt count, timestamps)
- Build a replay tool: after fixing the bug, replay DLQ messages back to the main topic
- Alert on DLQ size crossing a threshold

Q62 Explain backpressure in streaming systems.

Messaging

Backpressure is a mechanism for a consumer to signal to a producer that it cannot keep up with the current rate, causing the producer to slow down.

The problem without backpressure

Producer (fast) → Consumer (slow). Without backpressure: messages queue up unboundedly, memory fills up, consumer crashes, or messages are dropped.

Backpressure mechanisms

- Kafka (pull-based): consumers pull at their own rate. Natural backpressure — consumer controls its consumption rate.
- Go channels with bounded buffer: channel of size N blocks the producer when full. Producer automatically slows down.

```
jobs := make(chan Job, 100) // buffer of 100

go func() {
    for _, j := range allJobs {
        jobs <- j // blocks when buffer full - backpressure applied
    }
    close(jobs)
}()

for i := 0; i < numWorkers; i++ {
    go func() {
        for job := range jobs { process(job) }
    }()
}
```

When you cannot apply backpressure

Load shedding: intentionally drop low-priority requests under overload. Better to return 503 to 5% of requests than to crash under full load and return 503 to 100%.

Q63 How do you ensure message ordering in a distributed messaging system?

Messaging

Global ordering is expensive; partial ordering is often sufficient.

Within a Kafka partition

Messages within a single partition are strictly ordered. Use this by choosing the right partition key — all events for an order use orderId as key, ensuring they land in the same partition.

Across partitions

No global ordering guarantee. Design to not need it, or use a single partition (sacrifices parallelism).

Sequence numbers / version fields

Include a monotonic sequence number in each message. Consumers can detect gaps or out-of-order delivery and buffer until the missing sequence arrives.

Idempotent consumers

```
// Only apply event if it is newer than what we have
if event.Version > currentVersion {
    applyEvent(event)
}
```

Inbox pattern

Consumer writes events to an inbox table (ordered by sequence), processes them in order from the table. Decouples the consume step from the process step — allows reprocessing in order even if Kafka delivers out of order across partitions.

Q64

What is database query plan caching and why can it cause problems?

Messaging

PostgreSQL prepares and caches query execution plans for parameterized queries to avoid replanning on every execution.

How it works

```
-- First call: planner analyzes statistics, generates a plan
SELECT * FROM orders WHERE tenant_id = $1 AND status = $2

-- After 5+ calls: PostgreSQL caches a generic plan
-- (not specific to any particular $1/$2 values)
```

The problem: stale plans

If the data distribution changes significantly (a new tenant with 10x more data), the cached plan may become suboptimal. A query that was fast suddenly becomes slow after data growth.

Symptoms

EXPLAIN ANALYZE shows a bad plan (sequential scan when an index scan would be better) despite the data being present in the index.

Solutions

- ANALYZE tablename: update statistics so the planner has accurate information
- SET plan_cache_mode = force_custom_plan: disables generic plan caching for that session
- Increase autovacuum frequency for hot tables so statistics are always fresh
- PgBouncer in transaction mode: prepared statements are not cached across transactions (avoids stale plans but loses planning performance benefit)

What is the difference between microservices synchronous and asynchronous communication and when do you use each?

(See the Software Design section for synchronous vs asynchronous communication. This entry adds nuance around event-driven messaging specifically.)

Choreography vs Orchestration in async systems

- Choreography: each service listens to events and decides its own reaction. Low coupling. Hard to see the overall flow as complexity grows.
- Orchestration: a central orchestrator service directs each service and tracks the overall workflow. More explicit, easier to debug, but the orchestrator becomes a dependency.

Exactly-once vs at-least-once in practice

Design for at-least-once delivery + idempotent consumers rather than chasing exactly-once at the infrastructure layer. Idempotent consumers give you the same outcome with far less complexity.

Schema evolution in event-driven systems

- Use a schema registry (Confluent Schema Registry with Avro) to enforce compatibility
- Backward compatible changes only: add optional fields, never remove or rename
- Versioned event types: OrderPlacedV1, OrderPlacedV2 when breaking changes are unavoidable

Quick Reference: Key Concepts at a Glance

Concept	One-Line Summary
REST idempotency key	Client-generated UUID; server caches response; enables safe retries
Cursor pagination	WHERE id > last_seen ORDER BY id LIMIT N — stable, O(log n), no skips
Circuit breaker states	CLOSED → OPEN (on failures) → HALF-OPEN (probe) → CLOSED (on success)
Expand-contract migration	Add new column → write to both → backfill → switch reads → drop old
1NF / 2NF / 3NF	Atomic values / No partial PK deps / No transitive deps
CAP theorem	CP (consistent but unavailable during partition) vs AP (available but stale)
Saga vs 2PC	Saga = local txns + compensations (eventual); 2PC = blocking distributed lock
Outbox pattern	Write event to DB in same txn as business data; poller publishes to Kafka
SOLID — D principle	Depend on abstractions (interfaces), not concretions (concrete types)
p99 vs average latency	Fan-out of 100 services: if each has 1% slow chance, 63% of requests hit a slow one
Cache stampede fix	Mutex on cache miss + probabilistic early expiration
DLQ purpose	Capture messages that fail all retries; enable replay after bug fix
Backpressure in Go	Bounded channel (make(chan Job, N)) blocks producer when consumer is slow
Hedged requests	If no reply by p95, send duplicate to second instance; accept first reply
CQRS	Separate write model (normalized, consistent) from read model (denormalized, fast)
Liveness vs readiness	Liveness: is process alive? Readiness: is it ready for traffic?
Token bucket rate limit	Bucket of N tokens refilled at rate R; bursts up to bucket size allowed
Graceful shutdown	SIGTERM → stop accepting → finish in-flight → close resources → exit
Modular monolith	Strict module boundaries in one codebase; extract services when justified